UNIVERSITY MALAYSIA TERENGGANU
FACULTY OF COMPUTER SCIENCE & MATHEMATICS

SOFTWARE ARCHITECTURE
CSE3433

Group No. 11

SERVICE-ORIENTED SMART CAMPUS SUPPORT SYSTEM

Prepared by:
1. SARAH AMIRA HERI (S75812)
2. ELSA FADLIN BINTI MERSING (S75412)
3. SITI AISYAH ARWIAH BINTI YUNDI (S74256)

Prepared for:

DR. MOHAMAD NOR HASAN

[BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS]
SEMESTER 2 2025/2026

Table Of Content

1.0 System Overview

The Smart Campus Support System is a centralized digital platform designed to improve and streamline campus services for students and staff. The system is developed using Service-Oriented Architecture (SOA), where different services such as user management, facility booking, complaint handling, and notifications function as independent modules but are connected through APIs.

This system integrates multiple campus services into a single platform, allowing users to access various functions through one interface instead of using separate systems. Each service focuses on a specific task while still being able to communicate with other services when needed. This modular design makes the system more flexible, scalable, and easier to maintain.

The system supports different types of users, mainly students and administrative staff, with different access levels. Students can register, manage their profiles, book facilities, submit complaints, and receive notifications. Meanwhile, administrative staff can manage user data, monitor complaints, and send announcements more efficiently.

In addition, the facility booking module allows users to check availability and make reservations in real time, which helps reduce scheduling conflicts. The complaint management feature also enables users to report issues and track their status, ensuring faster response and better transparency.

Overall, the Smart Campus Support System aims to replace manual and disconnected processes with a unified, efficient, and user-friendly solution that improves productivity and enhances the overall campus experience.

2.0 Functional Requirements

The system will include the following main functionalities:

1. User Management

- Students can register and log in
- Profile management

2. Facility Booking

- View available facilities (e.g., rooms, labs, halls)
- Book and cancel reservations

3. Complaint Management

- Submit complaints or issues
- Track complaint status

4. Notification System

- Receive announcements from administration
- Get alerts for booking status or complaint updates

5. Service Integration

- All services are connected through SOA
- Each module works independently but communicates with others

3.0 Non-functional Requirements

Non-functional requirement define how the system should operate

i. Performance
   - The system should load pages quickly.
   - Communication between services is fast using lightweight APIs.
   - Latency between modules is minimized.

ii. Reliability
   - Each service operates independently; this ensures one service does not affect the overall system functionality.

iii. Scalability
   - Booking service can scale more during peak times.
   - Complaint service can scale independently if many users submit issues.

iv. Interoperability
   - Standardized communication protocols such as RESTful APIs enable seamless interaction between services.

v. Maintainability
   - System can be improved through a loosely coupled architecture.

4.0 Architecture Style Explanation

For this system, the selected architecture style is Client-Server Architecture, supported by Representational State Transfer (REST) principles.

Client-Server architecture is a model where the system is divided into two main components which are the client and the server. The client refers to the user interface (such as a web browser or mobile app) that users interact with, while the server is responsible for processing requests, managing data, and handling system logic.

In the Smart Campus Support System, students and administrative staff act as clients who send requests to the server, such as logging in, booking facilities, submitting complaints, or viewing notifications. The server processes these requests and returns the appropriate responses. This structure allows centralized control of data and services, making the system more secure and easier to manage.

To enhance communication between the client and server, the system applies REST (Representational State Transfer). REST uses APIs to enable smooth interaction between different modules and services. Each function in the system, such as user management or booking, can be accessed through specific API endpoints. This makes the system more flexible and allows different services to communicate efficiently.

This architecture is chosen because it supports a centralized system, which is one of the main objectives of the project. It also improves performance, scalability, and maintainability. For example, updates can be made on the server without affecting the client side, and multiple clients can access the system at the same time.

5.0 Service or Component Identification

Service Oriented Architecture (SOA) are an architecture design pattern where different independent or loosely coupled services are used to create a complete application

1. User Management Service
    a. Responsibilities: user registration, profile management and role management
    b. Tech used : Keycloak

2. Facility Booking Service
    a. Responsibilities: check facilities availability, book/cancel availability and prevent any overlapping booking
    b. Tech used : Node.js

3. Complaint Management Service
    a. Responsibilities: submit complaint and track them
    b. Tech used: Springboot

4. Notification Service
    a. Responsibilities: send notification
    b. Tech used: RabbitMQ

5. Inter-Service Communication
    a. Responsibilities: act as a communicator in between services
    b. Tech used: REST APIs and Apache Kafka

6.0 Architecture Diagrams

6.1 System Context Diagram

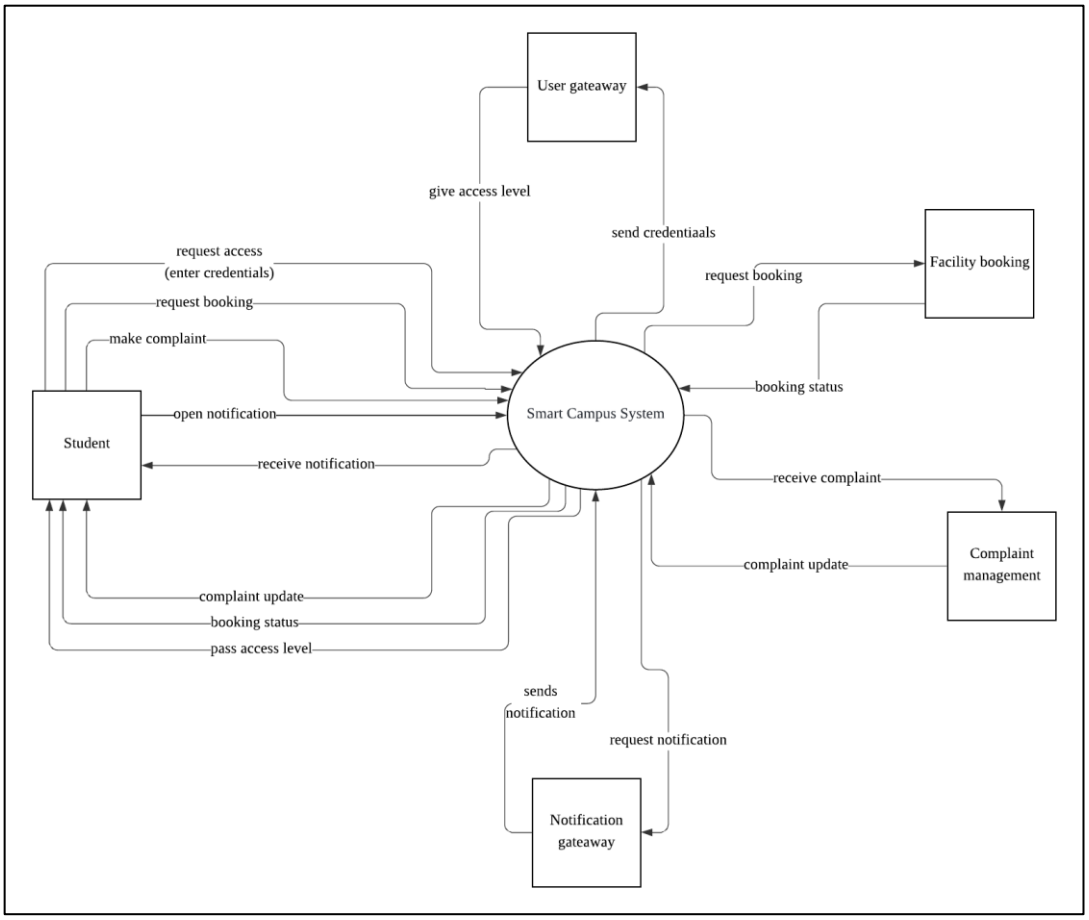

Figure 1. System Context Diagram

Figure 1 shows the System Context Diagram of the system. Student is the main user of the system. They'll use the system by requesting modules like access by entering their credentials, booking and make complaint or open their notification. This action will send their request to the system and the system will forward their request to the responsible 3rd party services. For example: when student want to book a facility they'll request the booking module in the system. This request will be forwarded by the main system to an external faculty booking services. This services then will send back booking status to the main system and the system will forward booking status to the student booking.

6.2 Component/Service Architecture Diagram

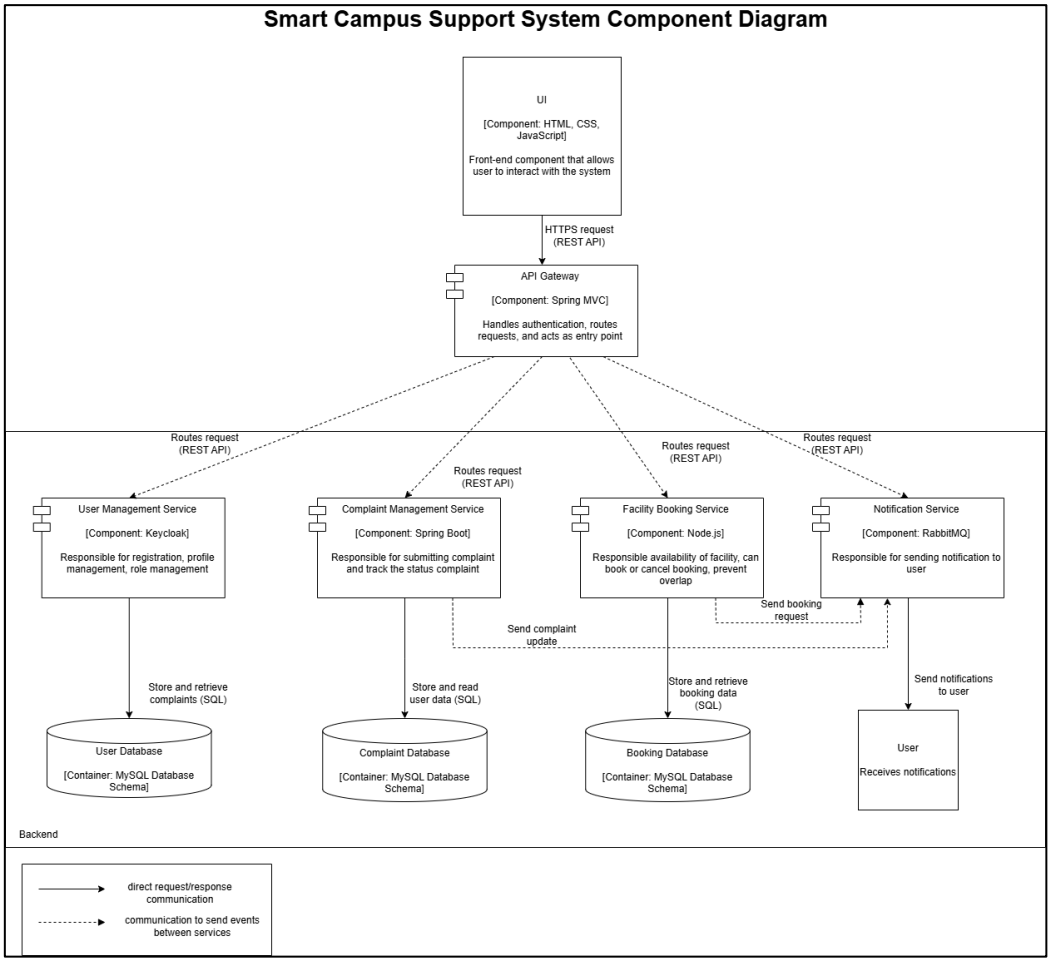

Figure 2. Component/Service Diagram

The system is designed using Service-Oriented Architecture, where each component is loosely coupled. The User Management Service handles authentication and user data, while the Facility Booking Service manages reservations. The Complaint Management Service handles issue reporting, and the Notification Service delivers system alerts. The API Gateway handles all incoming requests, which are routed to independent services such as User, Booking, and Complaint services via REST APIs. Each service maintains its own database. Event-driven communication allows services to send updates to the Notification Service, which delivers messages asynchronously to users.

6.3 Interaction Diagram

6.3.1 Booking Module Interaction Diagram

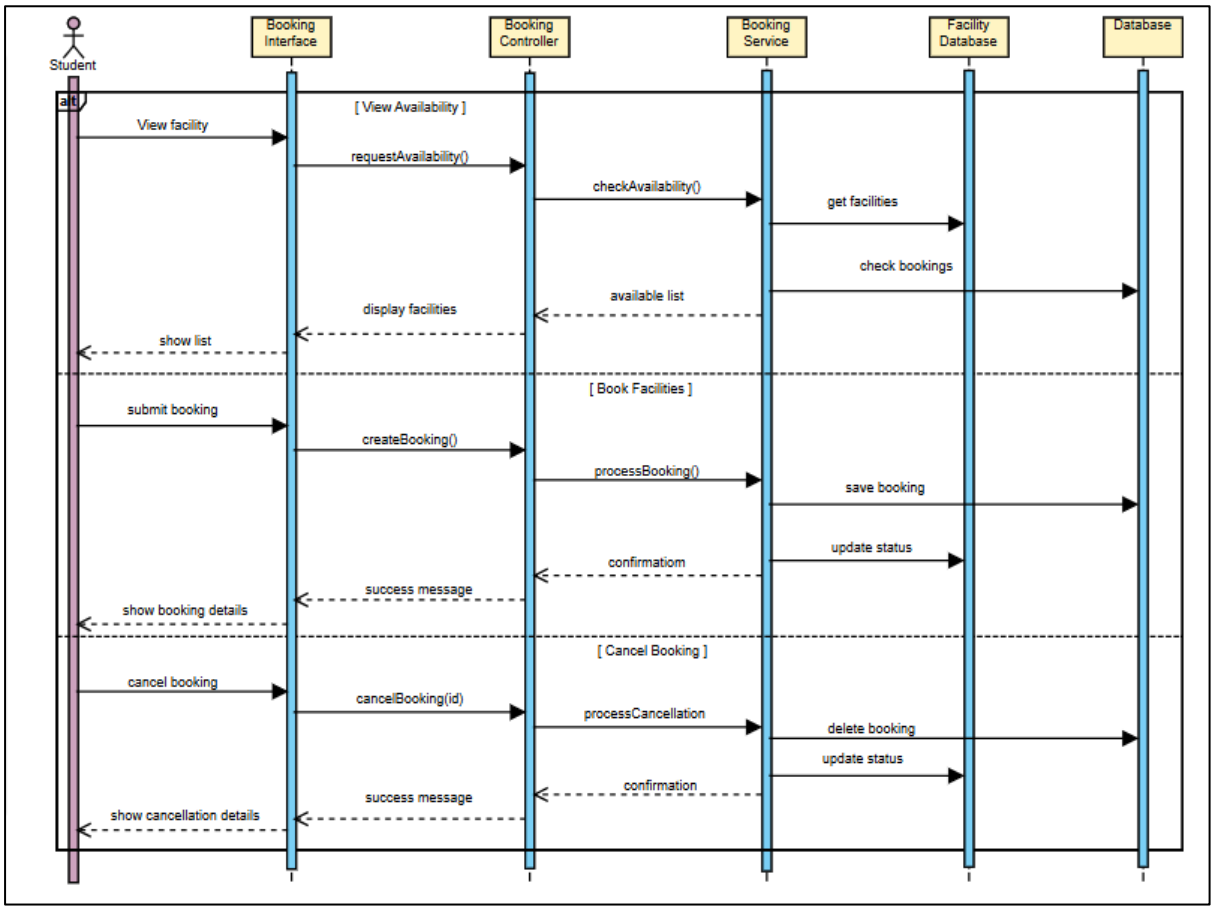

FIgure 3. Booking Module Interaction Diagram

This interaction diagram shows how the Facility Booking system manages viewing facilities, booking, and cancellation using a simple flow. The student interacts with the booking interface, which sends requests to the controller and service layer for processing. The system checks availability, saves booking details, or cancels reservations while updating the facility and booking databases accordingly. The results are then returned to the student as confirmation or updated information. This diagram keeps the structure simple while still fulfilling the required system components.

6.3.2 Complaint Module Interaction Diagram

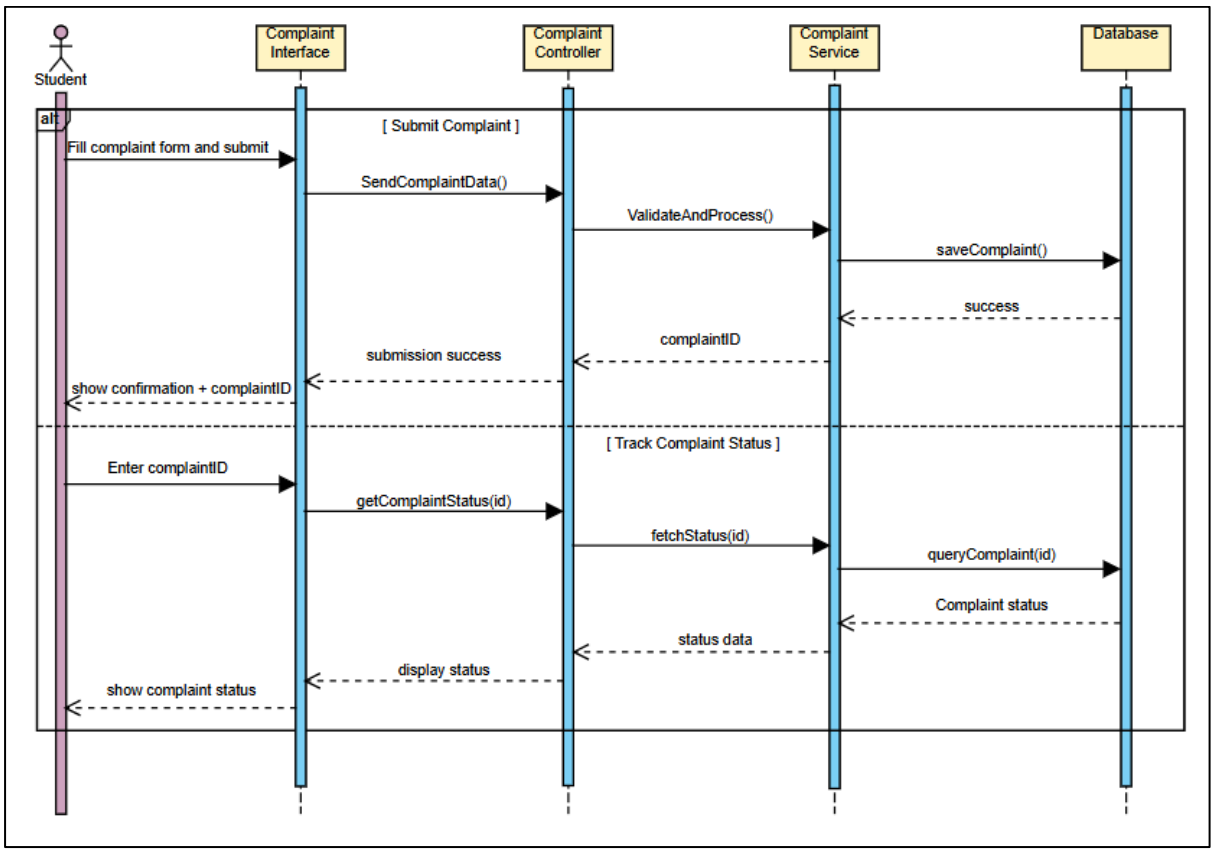

Figure 4. Complaint Module Interaction Diagram

The interaction diagram shows how the Complaint Management system handles two main functions which are submitting a complaint and tracking its status. When a student submits a complaint, the information is sent through the interface to the controller and service layer, where it is processed and stored in the database, and a confirmation along with a complaint ID is returned to the student. For tracking, the student enters the complaint ID, and the system retrieves the complaint details from the database through the same layers before displaying the current status. This diagram highlights how the system components work together to manage both processes efficiently in a single flow.

6.3.3 Notification Module Interaction Diagram

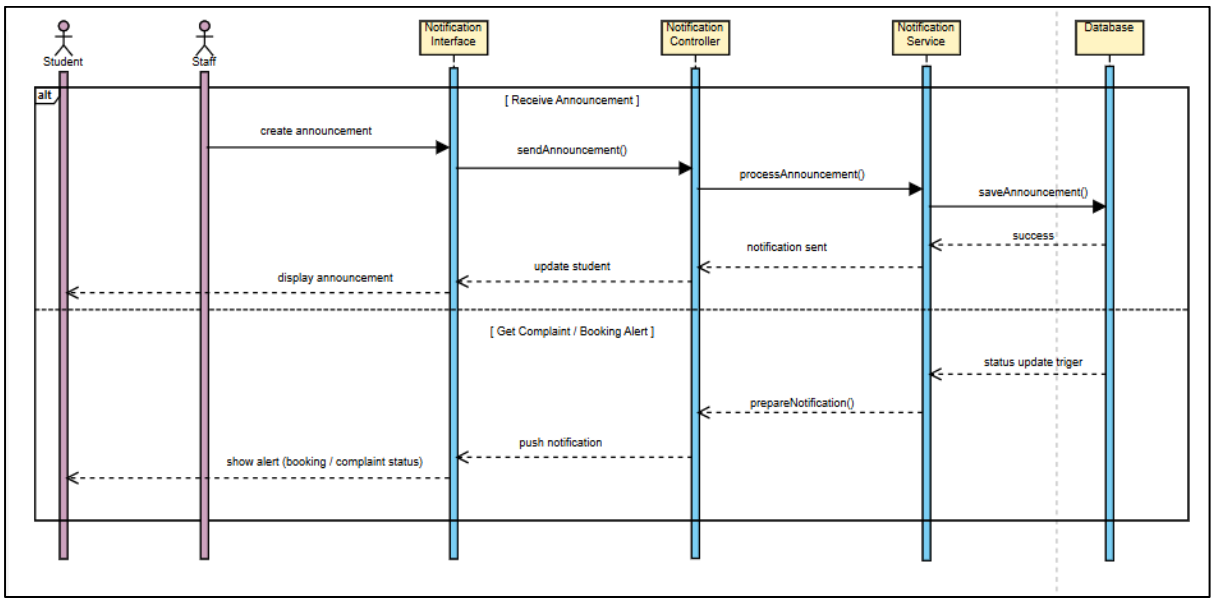

Figure 5. Notification Module Interaction Diagram

This interaction diagram shows how the Notification System manages both announcements and alert notifications. For announcements, the staff creates a message through the interface, which is processed, stored in the database, and then delivered to student. For alerts, the system automatically detects updates such as booking status or complaint progress from the database and sends notifications to student through the controller and interface. Overall, the diagram illustrates how the system ensures student stay informed by handling both manual announcements and automatic alerts in a single flow.

6.4 Deployment Diagram

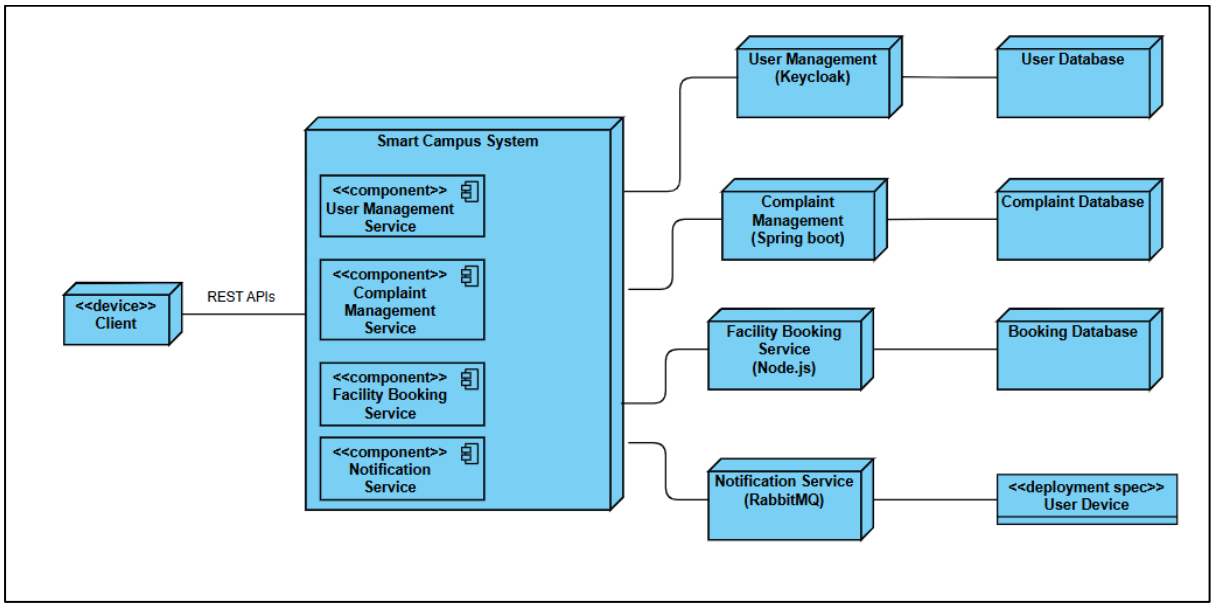

Figure 6. Deployment Diagram

Figure 6 shows the Deployment Diagram of the system. Client will access the system via REST APIs. The system is made up of 4 components and each of the components in the system will call on the 3rd party services to do the components function. For example: the notification service in the system will call out on RabbitMQ to send notification to the user.